f1r3|@ng

DESIGN NOTE · DRAFT 3

Monadic types and the foreach comprehension

A synchronous comprehension over declared data monads, with programmable aggregation and explicit cost

L.G. Meredith · F1R3FLY.io · October 2026

THE IDEA

TWO COMPREHENSIONS

for binds in the space

```
for( ptrn ← chan where cond ) P
```

Sources are channels, written by concurrent processes. When several candidates match, one of them is chosen and the others are not.

foreach binds in a data monad

```
foreach( ptrn ← mndInst where cond )
  yield( rslt ) P
```

The source is a term of a monadic sort, and there is no writer to meet. Every candidate contributes, and one result is published on rslt.

The two share patterns, per-group where clauses and the use of ; for sequence. They differ only in where binding happens: choice in the space, aggregation in the monad.

SYNTAX

THE COMPREHENSION, PART BY PART

```
foreach( ptrn ← mndInst where cond ) yield( rslt ) by A P
```

part	what it does
ptrn <- mndInst	visits every element of a term whose sort carries a declared monad
where cond	filters; a rejected candidate contributes the aggregator's unit
yield(rslt)	names the channel on which the single result is published
by A	names the aggregator, a theory instance; the default is ParMonoid
P	the body, evaluated to normal form by lookahead

SEMANTICS

MEANING IN ONE RULE

```
foreach( i ← { 1, 3, 5, 7, 9 } where i > 4 ) yield( rslt ) i    ⇝  rslt!( 5 | 7 | 9 )
foreach( i ← { 1, 3, 5, 7, 9 } where i > 4 ) yield( rslt ) i * i  ⇝  rslt!( 25 | 49 |
81 )
foreach( i ← { 1, 3 } where i > 4 ) yield( rslt ) i                ⇝  rslt!( 0 )
```

Bodies run

Each surviving body is reduced to normal form. During that lookahead it cannot meet anything outside itself.

The result is data

The publication is a send, so its payload is quoted. A consumer that wants the contributions to run says so.

One step

From outside, the comprehension is a single reduction whose reduct is a single output.

DATA DEFINITION SUBLANGUAGE

DECLARING A DATA MONAD

```
Monads {
  Seq over List {
    unit    : (a)              ⇝ [a];
    flatMap : ([], ^x.k)       ⇝ [];
    flatMap : ([a, ...r], ^x.k) ⇝ (Concat (subst ^x.k a) (flatMap r ^x.k));
    toBag   : ([a, ...r])      ⇝ {a, ...(toBag r)};
    ordered;
    laws    : LeftUnit, RightUnit, Assoc, ToBagMorphism;
  }
}
```

Rewrites, not code

unit and flatMap are given by cases over the sort's constructors, as ordinary rewrites.

Laws are obligations

They are proved or checked, never handed to the matcher, so matching avoids higher-order unification.

toBag is the interface

A monad morphism into finite bags: the elements the comprehension visits, with multiplicity.

DATA DEFINITION SUBLANGUAGE

WHAT FOREACH NEEDS FROM A MONAD

entry	meaning	licenses
toBag (required)	elements with multiplicity; a monad morphism	foreach over the monad, and its measure
ordered	a canonical order on the elements	ordered, non-commutative aggregators
commutative	independent binds commute	& under an ordered aggregator

measure(m) = |toBag(m)|

The completion condition of the reference semantics, and the quantity every cost estimate rests on.

No longer required

A zero, because filtering uses the aggregator's unit; an algebra on processes, because results are data.

LOOKAHEAD

THE TERMINATING FRAGMENT

Guards and bodies are reduced to normal form in isolation. A typed source calculus supplies termination, and its type derivation is the decidable certificate.

translate	map	lands in
terms	$M \mapsto [\![M]\!]$	a process in f1r3lang
types	$\tau \mapsto \phi\tau$	a spatial-behavioural formula implying termination

First: System F

Strongly normalising and Church–Rosser. Declared as a theory; guards and bodies are its terms, with explicit annotations.

Second: Abramsky's proof expressions

Terminating and confluent. A body may communicate internally on linear channels and still have one normal form.

LOOKAHEAD

A FAITHFUL TRANSLATION

Divergence is reflected

Each step of ⟦M⟧ tracks zero or more steps of M, with finitely many administrative steps between.

Confluence transfers

One reduction strategy is fixed, so a confluent source yields unique normal forms.

The formula map is sound

$\vdash M : \tau$ implies $[\![M]\!] \models \phi\tau$, and $\phi\tau$ implies termination, $\mu X.\Box X$.

Ground values read back

A body typed Int normalises to an integer the aggregator can combine.

Termination, not a bound. A comprehension that is not sufficiently funded does not complete, publishes nothing, and its deployer is notified.

SEMANTICS

THE REFERENCE TRANSLATION

```
[[ foreach( ptrn ← m where cond ) yield( rslt ) P ]] =
  new src, acc, ans in {
    [[ m ]]_src
  | acc!( 0, Nil )
  | if ( n == 0 ) { rslt!( Nil ) }
  | for( @x ≤ src ){
      match x {
        ptrn where cond ⇒ ans!( [[ P ]] )[*]
        _               ⇒ ans!( Nil )
      }
    }
  | for( @(k, a) ≤ acc & @e ≤ ans ){
      if ( k + 1 == n ) { rslt!( a | e ) }
      else { acc!( k + 1, a | e ) }
    }
  }
```

Initialisation

The private accumulator starts at the unit.

Completion

rslt is written once, when the count reaches n = measure(m). No partial sum is ever visible.

Every element counts

A failed pattern or guard still contributes the unit.

AGGREGATION IS PROGRAMMABLE

PPar is HashBag(Proc), so parallel composition is the free commutative monoid. Every other aggregator is defined as the unique homomorphism out of it.

```
ĥ(P | Q) = ĥ(P) ⊕ ĥ(Q)     ĥ(0) = ε
foreach( i ← { 1, 3, 5, 7, 9 } where i > 4 ) yield( rslt ) by a.IntSum() i  ⇝  rslt!(
21 )
```

No new semantics

Parallel composition stays the reference meaning.

Fusion is a theorem

Folding as contributions arrive is correct because ĥ is a homomorphism.

Order stays invisible

It holds for every commutative aggregator, as for par.

AGGREGATION

AGGREGATORS ARE THEORY INSTANCES

```
Theory IntSum() {
  let s  = u.EmptySet() in (
  let m  = u.Monoid(s) in (
  let cm = u.CommutativeMonoid(m) in (
  cm
    Exports      { Elem ⇒ Int; }
    Replacements { Zero ⇒ IZero . ⊢ "0" : Int;
                   Plus ⇒ IAdd  . x:Int, y:Int ⊢ x "+" y :
Int; }
    Rewrites     { IAddEval : (IAdd n k) ⇝ [n + k]; }
  )))
}
```

Checked by origin

The operation must be the Plus of CommutativeMonoid, carried by its origin token, not merely named Plus.

Two diagnostics

aggregator-not-commutative, and aggregator-ambiguous for instances such as rigs.

Fixed at compile time

Instances elaborate in the client toolchain; the default is ParMonoid.

AGGREGATION

THE STAGED PATH

stage	aggregator descends from	examples	needs from the source
1	CommutativeMonoid	par, sums, counts, products	toBag
2	Semilattice	union, maximum, boolean or	toBag
3	Monoid only	lists, strings, composition	toBag and ordered
4	a V-Semimodule	weighted bags, amplitudes	toBag and a graded where

Each stage adds one algebraic requirement, checked by origin and costed by the same per-combine rule. Deduplicating a Set is a choice of aggregator, not of source.

SEQUENCING

FOREACH ON THE LEFT OF ;

```
foreach( p ← m where c ) yield( r ) P ; K
  ==  new r in { [[ foreach( p ← m where c ) yield( r ) P ]] | for( @_ ← r ){ K } }
```

x!?(Q); K

K runs only if the receiver on x chooses to reply. The guarantee rests on a convention.

foreach(...) yield(r) P ; K

Typing guarantees termination, so publication is certain whenever the run is funded.

The acknowledgement and the result are the same datum: `| for( @ys <- r ){ K }` binds it.

COST

TWO GATES, EXPLICIT COST

```
{% foreach( ptrn ← mndInst where cond ) yield( rslt ) by A {% P %}[t] %}[s]
```

gate	funds	estimate
outer, s	every binding, each combination, one publication	$n + (n - 1)(1 + c_{\oplus}) + 1$, exact given n
inner, t	the lookahead of each surviving body	$n \cdot dt$, the budget the deployer chooses

Not sufficiently funded: the comprehension does not complete, publishes nothing, and the deployer is notified through the budget trace.

WORKED EXAMPLE

THE PARAMETRIC RHO CALCULUS

A syntax monad

In RhoD(V), unit is the drop *v and flatMap is semantic substitution. It is declared by naming PDrop as the variable former, so its laws are discharged once, for the toolchain.

Orientation is a distributive law

OrientG is Rho after FrameG, with λ(g, P) = g · P. Beck's four laws are lemmas already proved in Oriented Processes.

Symmetry reduces precharge

One lookahead per Stab(P)-orbit of holes. By orbit counting, the number is measure(P) / |H|, and it is an integer.

Every communication performs one bind in the syntax monad: RComm substitutes the received code for the bound variable. The two comprehensions meet at the witness.

IMPLEMENTATION

NATIVE, WITH THE TRANSLATION AS SPECIFICATION

aspect	position
Semantics	the native single rule; foreach is a conservative extension of the core
Correctness	graded bisimilarity to the reference translation: same behaviour, same bill
Reuses	the spatial matcher and where guards on match cases with fall-through
Must add	the per-binding charge, the lookahead budget, and a demand rule
Representation	a new process node carrying binders, gated on the protocol version
Replay	one publication; lookahead and accumulation are recomputed

PLAN

THE FIRST TRANCHE

area	scope
Monads	Seq, Bag and the finite powerset in a prelude, each with toBag
Comprehension	<-, ; and &, crisp where, yield(rslt), and use before ;
Fragment	System F declared as a theory; guards and bodies are its terms
Aggregation	stage 1, by a CommutativeMonoid instance, default ParMonoid
Cost	both gates, per-binding outer charge, n-ary outer gate
Node	native rule, protocol-gated node, demand rule, obligations recorded

Next: Abramsky’s proof expressions as the second fragment. Deferred: sort parameters, aggregation stages 2 to 4, graded where in foreach.

PLAN

OPEN QUESTIONS

The logic of the fragment

Should the generated logics gain least fixed points and quantification over formulae?

Element sorts in System F

Import a theory’s sorts as base types, or represent them by Church encodings?

Dependent groups

Take outer gates group by group as the count unfolds, or a declared bound at entry?

Per-element funding

May the inner signature mention pattern variables, so that data pays for itself?

QUESTIONS

f1r3|@ng